西南交通大学 计算机与人工智能学院
School of Computing and Artificial Intelligence,Southwest Jiaotong University

试卷

考试名称： 高级语言程序设计2025第2次机考C卷 **院系：** **专业：** **姓名：** **学号：**

名称	分数
编程题 1 (Q33)	语义匹配分数：0，动态测试得分：20，最终得分：20
编程题 2 (Q34)	语义匹配分数：8.94，动态测试得分：20，最终得分：20
编程题 3 (Q35)	语义匹配分数：9.31，动态测试得分：20，最终得分：20
编程题 4 (Q36)	● 语义匹配分数：14.73，动态测试得分：20，最终得分：20
编程题 5 (Q37)	语义匹配分数：16.83，动态测试得分：20，最终得分：20
总分	100
备注	系统对于标为●的题目评分有异议，分数待定，需要教师人工审核后确定最终分数。

编程题（100分）

1.(Q33, 20分)根据最大公约数的如下3条性质，采用迭代法编写计算最大公约数的函数Gcd()，在主函数中调用该函数计算并输出从键盘任意输入的两整数的最大公约数。
性质1 如果a>b，则a和b与a-b和b的最大公约数相同，即Gcd(a, b) = Gcd(a-b, b)
性质2 如果b>a，则a和b与a和b-a的最大公约数相同，即Gcd(a, b) = Gcd(a, b-a)
性质3 如果a=b，则a和b的最大公约数与a值和b值相同，即Gcd(a, b) = a = b

要求如下：
（1）从键盘任意输入的两整数
主函数调用Gcd()函数，并输出两整数的最大公约数。
（2）Gcd函数原型为：
int Gcd(int a, int b);
如果输入的数不是正整数，则返回-1，否则，返回两个数的最大公约数。
（3）**输入提示信息格式要求："Input a,b:"
输入格式："%d,%d"
**输出提示信息要求：
如果输入的数不是正整数，则输出"Input error!\n"，否则按如下格式输出"Gcd=%d\n"

```c
#include<stdio.h>

int Gcd(int a,int b)
{
    if(a<=0||b<=0) return -1;
    else
    {
        if(a>b) return Gcd(a-b,b);
        else if(b>a) return Gcd(a,b-a);
        else return a;
    }
}

int main(void)
{
    int m,n;
    int shu;
    printf("Input a,b:");
    shu=scanf("%d,%d",&m,&n);

    if(shu!=2) printf("Input error!\n");
    else
    {
        int jieguo=Gcd(m,n);
        if(jieguo==-1) printf("Input error!\n");
        else printf("Gcd=%d\n",jieguo);
    }

    return 0;
}
```

2.(Q34, 20分)删除动态升序数组中的重复元素
（1）用指针申请一组动态空间（大小为一维n个元素的整型数组），n由用户进行输入，如果数组中的某个元素与排在它之后的元素重复，则对其进行删除，直到数组中没有重复元素为止。保证剩余元素的相对次序保持不变，打印删除重复元素后的数组。

首先输入n值，输入格式为"%d"，然后依次输入动态数组中的n个元素，元素按照升序方式输入，输入格式为“%d”，依次输出删除重复元素后的数组中的每个元素，输出格式为“%d ”，最后进行释放动态空间
输入样例：
输入n值提示信息："input n:" 输入格式"%d"
输入动态数组元素值提示信息："input %d data:" 输入格式"%d"
输入样例：
14 0 1 1 2 3 4 4 5 5 6 7 8 9 9
输出样例：
0 1 2 3 4 5 6 7 8 9

```c
#include<stdio.h>
#include<stdlib.h>

int main(void)
{
    int n;
    printf("input n:");
    scanf("%d",&n);

    int *p=(int *)malloc(n*sizeof(int));
    int *pflag=(int *)malloc(n*sizeof(int));

    int i;
    printf("input %d data:",n);
    for(i=0;i<n;i++)
    {
        scanf("%d",p+i);
        pflag[i]=1;
    }

    for(i=0;i<n;i++)
    {
        if(p[i]==p[i+1])
        {
            pflag[i]=0;
        }
    }

    for(i=0;i<n;i++)
    {
        if(pflag[i]) printf("%d ",p[i]);
    }

    free(p);
    free(pflag);
    return 0;
}
```

3.(Q35, 20分)链表完成成语接龙。

链表结构

```
typedef struct Node {
 char str[9];
 struct Node *next;
} NODE;
```

（1）定义NODE * CreateOrderLink(NODE *head)函数，实现输入成语完成接龙链表。

程序先打印信息"请输入第一个4字成语\n"，

然后用%s格式读入一个汉语成语；

程序接着打印信息"请再输入一个4字成语\n",

再用%s格式读入另一个汉语成语，接下来需要判断前一个成语的最后一个汉字与后一个成语的第一个汉字是否相同，若相同（满足成语接龙条件），则程序需要不断重复打印信息"请再输入一个汉语成语\n"，然后用%s格式读入一个新的成语，直到新输入的成语的第一个汉字与前一个成语的最后一个汉字不相同时，则停止输入，在打印"接龙结束。\n"后（2）定义void PrintLink(NODE *head)函数，输出接龙成语链表中接龙成语。

要求： 首先打印信息"打印接龙成语：\n"

然后每一个节点成语的输出"%s\t"；

（3）定义int compare(char *s, char *t)来判断前后两个成语(s指向前一个成语，t指向后一个成语)是否满足接龙条件。例如: compare("叶公好龙", "龙马精神") 的返回值为1(满足接龙条件), 而compare("一目十行", "一望无际")的值为0(不满足接龙条件)。

提示：一个汉字的编码占用char数组的两个元素； 假定成语字符串的长度不超过4个汉字（即串长最大为8个字节)。4字成语的第一个汉字为str[0]和str[1]、最后一个汉字为str[6]和str[7]。

（4）main()如下：

```
int main() {
 NODE *head = NULL;
 head = CreateOrderLink(head);
 PrintLink(head);
 return 0;
}
```

程序的一次运行示例如下。

```
请输入第一个4字成语
叶公好龙
请再输入一个4字成语
龙马精神
请再输入一个4字成语
神采飞扬
请再输入一个4字成语
洋洋得意
接龙结束。
打印接龙成语：
叶公好龙        龙马精神        神采飞扬
```

```c
#include<stdio.h>
#include<stdlib.h>

typedef struct Node {
    char str[15];
    struct Node *next;
} NODE;

NODE * CreateOrderLink(NODE *head)//差点把这个返回一个指针看漏了。
{
    head=(NODE *)malloc(sizeof(NODE));
    NODE *pLast=head;

    printf("请输入第一个4字成语\n");
    scanf("%s",pLast->str);

    while(1)
    {
        printf("请再输入一个4字成语\n");
```

```
        NODE *pNew=(NODE *)malloc(sizeof(NODE));
        scanf("%s",pNew->str);

        int si=compare(pLast->str,pNew->str);
        if(si==1)
        {
            pLast->next=pNew;
            pNew->next=NULL;
            pLast=pNew;
        }
        else
        {
            free(pNew);
            break;
        }

    }
    printf("接龙结束。\n");
    return head;
}

void  PrintLink(NODE *head)
{
    NODE *p=NULL;
    printf("打印接龙成语：\n");

    for(p=head;p!=NULL;p=p->next)
    {
        printf("%s\t",p->str);
    }

}

int compare(char *s, char *t)
{
    if(s[9]==t[0]&&s[10]==t[1]&&s[11]==t[2]) return 1;//注意此处为汉字，两个字节。
    else return 0;
}

int main()
{
    NODE *head = NULL;
    head=head = CreateOrderLink(head);
    PrintLink(head);
    return 0;
}
```

4.(Q36, 20分)有如下3*4的矩阵，编写函数获取最大元素值对应的行列下标，并输出最值和行列下标。

```
 1  2  3  4
 9  8  7  6
10 -1 -4  4
```

要求（1）在main函数中完成数组的定义及数组数据的初始化，初始化数据值如上数据。
（2）编写函数 void maxData(),完成最大元素得行列下标的确定
（3）在主函数中完成 maxData的调用，输出最值和行列下标。

**要求输入提示信息为：无输入提示信息和输入数据。
**要求输出格式为："最大值=%d, 在%d行%d列\n"

```
#include<stdio.h>

void maxData(int shu[][4],int m,int n,int *a,int *b)
{
    int i,j;
    int max=shu[0][0];
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            if(shu[i][j]>max)
            {
                *a=i;
                *b=j;
                max=shu[i][j];
            }
        }
    }
}

int main(void)
{
    int shu[3][4]={{1,2,3,4},{9,8,7,6},{10,-1,-4,4}};
    int m=3,n=4;
    int a,b;
    maxData(shu,m,n,&a,&b);

    printf("最大值=%d, 在%d行%d列\n",shu[a][b],a,b);
    return 0;
}
```

5.(Q37, 20分)从键盘输入一个数字字符串（最长不超过80字符），用字符数组编程判断该数字字符串对应的数字共有几位数字（忽略字符串中包含的正负号和非数字字符），并输出字符串中包含的各个数字的个数。要求按照如下函数原型编程。

```
//函数功能：返回字符数组str中包含的数字字符的总数
int DigitBit(char str[]);
//函数功能：统计并输出字符数组a中包含的各个数字的个数
void DigitCount(char a[]);
```

在主函数中先调用DigitBit()输出数字字符串中数字的位数，然后调用DigitCount()输出字符串中包含的各个数字的个数。
输入字符串的提示信息："Input a number:"
输入字符串的格式："%s"
输出格式和提示信息：
    输出共有几位数字用："%d bits\n"

输出各个数字对应的个数用："%d: %d\n"
程序运行结果示例1：
Input a number:12226↙
5 bits
1: 1
2: 3
6: 1
程序运行结果示例2：
Input a number: -12243abc↙
5 bits
1: 1
2: 2
3: 1
4: 1
注意：1）不要使用指针、结构体和goto语句编程。
2）请直接拷贝粘贴题目中给定的格式字符串。

```
#include<stdio.h>
#include<string.h>

int DigitBit(char str[])
{
    int i;
    int sum=0;
    for(i=0;str[i]!='\0';i++)
    {
        if(str[i]>='0'&&str[i]<='9')sum++;
    }
    return sum;
}

void DigitCount(char a[])
{
    int i;
    int number[10]={0};

    for(i=0;a[i]!='\0';i++)
    {
        if(a[i]>='0'&&a[i]<='9')
        {
            number[a[i]-'0']++;
        }
    }

    for(i=0;i<10;i++)
    {
        if(number[i]!=0)
        {
            printf("%d: %d\n",i,number[i]);
        }
    }
}

int main(void)
{
    char wen[1001];
    printf("Input a number:");
    scanf("%s",wen);

    int sum=DigitBit(wen);
    printf("%d bits\n",sum);

    DigitCount(wen);
    return 0;
}
```